

nano-case: A 1M-Parameter Transformer that Reads Identifiers a Regex Cannot Split

A Technical Report by Vuk Rosić

Subject: nano-case, an open-source (MIT) ~1M-parameter byte-level transformer for case conversion of boundary-destroyed identifiers, trained 100% on code-generated data.

Abstract

Converting a *clean* identifier between case styles — `snake_case`, `kebab-case`, `camelCase`, `PascalCase`, `CONST_CASE` — is a solved, free problem: a regular expression splits on the separators and case humps and re-renders. nano-case targets the slice a regex provably cannot handle: **boundary-destroyed** inputs where the separators and casing are gone (`sdkmodel`, `httprequesthandler`), leaving nothing to split on. The only route back to the intended words is a learned vocabulary. On a held-out test set, nano-case reaches $99.8\% \pm 0.0\%$ exact-match overall (3 training seeds) versus 61.8% for a standard regex converter, and on the regex-killer ‘smushed’ slice (N=1410) $99.7\% \pm 0.0\%$ versus the regex’s 8.2% . The model has 1,016,960 parameters, runs on a CPU in milliseconds, and was trained entirely on data generated by code — no scraping, no labelling, no distillation. We also report where it breaks: its segmentation prior *is* its ~120-token training vocabulary, so it collapses on out-of-vocabulary words (2%) and on chains longer than seen in training (33%).

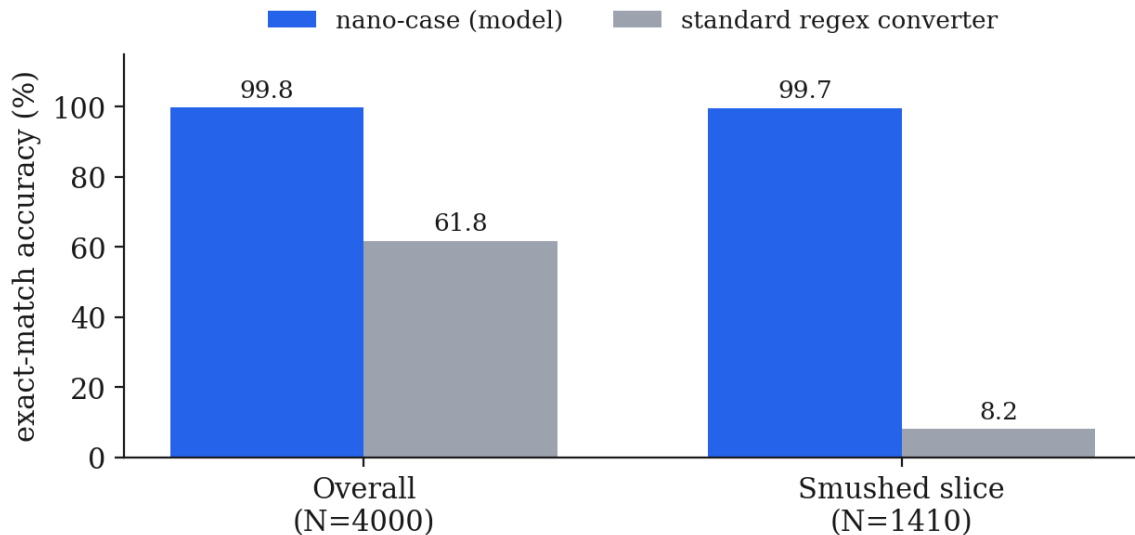


Figure 1. Exact-match accuracy, nano-case vs a standard regex case-converter. Left: overall held-out set (N=4000). Right: the ‘smushed’ regex-killer slice (N=1410) — boundary-destroyed, single-case, multi-word inputs. The 8.2% vs 99.7% gap is the ‘you genuinely need a model here’ result. Model bars are the mean over 3 training seeds.

1. The problem: when a script runs out of signal

A deterministic converter works by reading structure that is still present in the input. Given `userTableHandler` it splits on the case humps; given `user_table_handler` it splits on underscores. That is genuinely free, so building a model for it would be dishonest. The interesting case is when a developer (or an upstream tool) has *destroyed* those boundaries: an all-lowercase, separator-free token such as `userprofilecache`, or an all-caps `HTTPREQUESTHANDLER`. A regex has no signal left to split on — the word boundaries exist only in a reader’s knowledge of which character sequences are words. That knowledge is

exactly what a small language model can hold, and what a script cannot.

2. Data: correct by construction, generated entirely by code

There is no dataset to download. Each training example is synthesised: sample 1–4 words from a fixed ~120-token vocabulary (common code words, acronyms, digit tokens), render the **gold target** canonically in the chosen case style, then corrupt a copy of the input — about 45% of the time by destroying all boundaries (lower/upper smush). Because the target is rendered from the known word list, every label is correct *by construction*; there is nothing to mislabel. A held-out set uses a disjoint RNG seed, and a test checks its overlap with the 100k-example training set stays below 25%.

Training is supervised fine-tuning with the prompt masked: the loss is computed only on the target identifier and a one-byte newline end-of-sequence marker, never on the `<case> | <messy input> =>` prompt. The model is a ~1M-parameter byte-level decoder-only transformer (RMSNorm, RoPE, grouped-query attention, SwiGLU; dim 128, 4 layers), trained for 12k steps with AdamW and a cosine schedule.

3. Results

We evaluate exact-match accuracy on a held-out set (seed 987654321, N=4000) against `regex_convert`, a standard separator-and-hump case converter. Headline numbers are the mean $\pm$ standard deviation over three independent training seeds (0/1/2); the variance is below 0.05%, so it rounds to 0.0%.

	nano-case (model)	regex script
Overall (N=4000)	99.8% $\pm$ 0.0%	61.8%
Smushed slice (N=1410)	99.7% $\pm$ 0.0%	8.2%

Table 1. The regex scores 61.8% overall because the held-out set mixes easy (still-structured) inputs it can handle with hard (boundary-destroyed) ones it cannot. Restricting to the smushed slice isolates the hard case: the script collapses to 8.2% while the model holds at 99.7%.

4. Where it breaks (out-of-distribution)

An honest nano model reports its ceiling. nano-case’s segmentation ability *is* its training vocabulary, so we probe three out-of-distribution regimes against the in-vocabulary control.

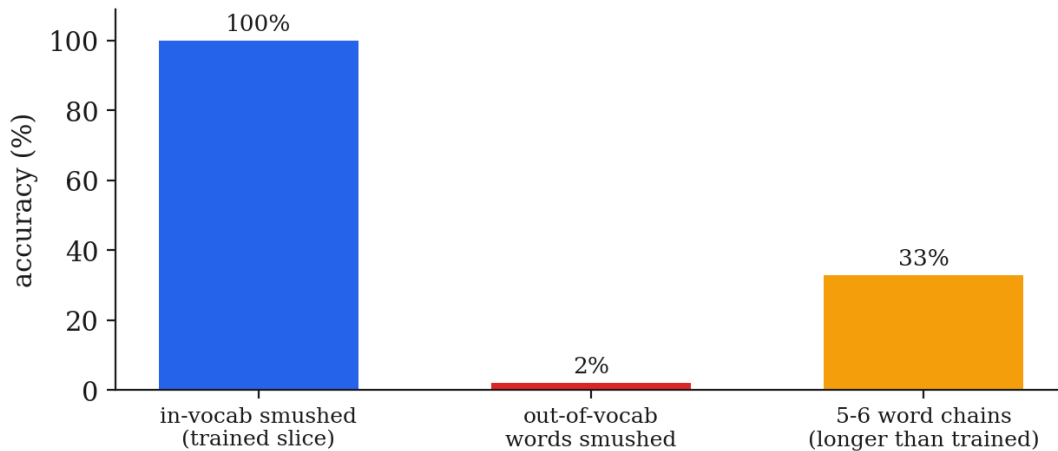


Figure 2. OOD breaking-point probe (seed-0 model, $N=400$ each). The model nails smushing of known words (100%), but has no segmentation prior for out-of-vocabulary tokens (2%) and degrades on chains longer than the 1–4 words seen in training (33%). This is the expected ceiling of a 1M-parameter model on a vocabulary task — reported, not hidden.

5. Scientific rigor and reproducibility

Every claim in this report is regenerated from the published weights. The release ships: a self-contained inference file (`modeling_nano_case.py`, torch + safetensors only); the data generator (`data_cases.py`); the benchmark (`eval_nano_case.py`); and a test suite (`test_nano_case.py`) that checks labels are correct by construction, train/test overlap stays low, decoding is deterministic, and — as a regression gate — that the shipped `model.safetensors` still performs the hard, regex-defeating conversions. Reproduce:

```
pip install -r requirements.txt
python eval_nano_case.py --n 4000 # model vs regex, overall + smushed slice
pytest test_nano_case.py -q      # labels / overlap / determinism / weights regression
```

Author's note

Independent technical report by Vuk Rosić, 30 June 2026. nano-case and the voidlab training harness are the author's own work, released under the MIT license. All figures are original plots of the model's own measured results; all numbers are reproduced from the published weights with the shipped benchmark. Any interpretation errors are the author's own.